Rust Network Architecture: Visual Diagrams & Charts

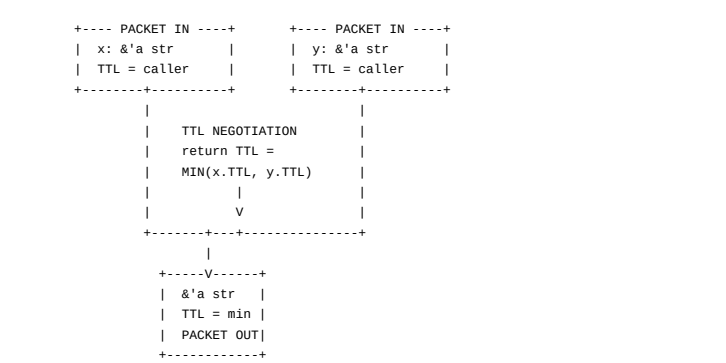
Architectural diagrams of Rust's ownership model mapped to network topology,
frame scheduling, lifetime protocol, and access control systems.

---

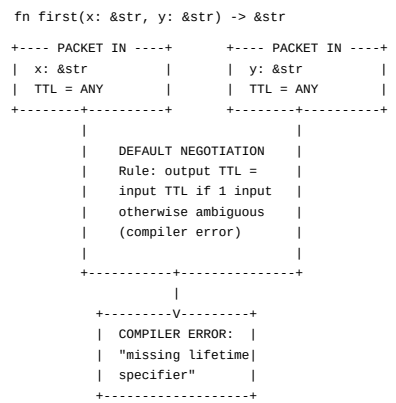
1. ARCHITECTURAL OVERVIEW: Rust as a Campus Network

---

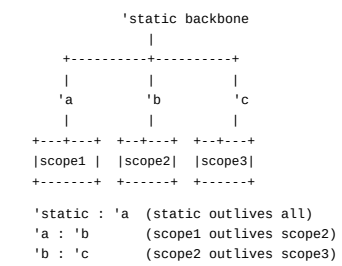
2. LIFETIME PROTOCOL: TTL Negotiation Diagram



ELISION PROTOCOL (when lifetimes are not written):



LIFETIME SUBTYPING ('a : 'b means 'a OUTLIVES 'b):

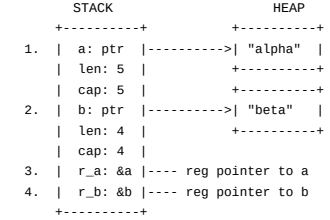



```
'static : 'a  (static outlives all)
'a : 'b       (scope1 outlives scope2)
'b : 'c       (scope2 outlives scope3)
```

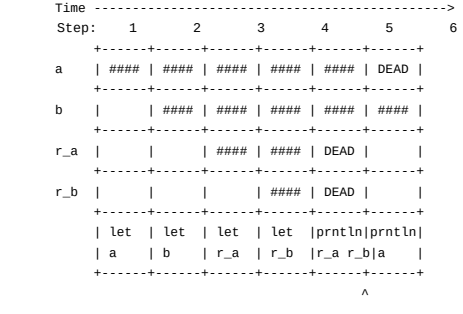
---

3. FRAME SCHEDULE: Stack Lifecycle Chart

FRAME ALLOCATION SEQUENCE:



LIVENESS TIMELINE:



```
                                    last use of a
                                    a becomes DEAD
                                    (drop scheduled)
```

DROP SCHEDULE (reverse declaration order at scope end):

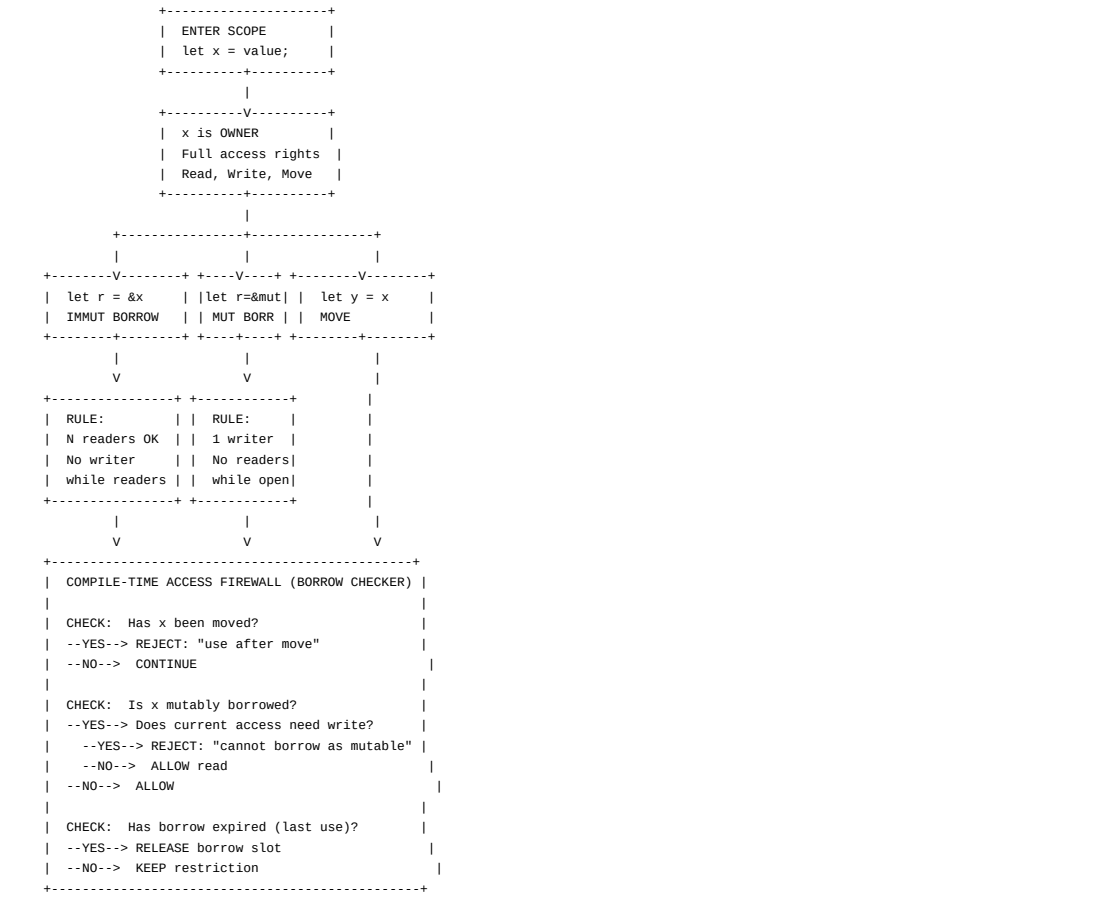
```
      DEAD ---- r_b (already dead, no-op)
      DEAD ---- r_a (already dead, no-op)
      ALIVE --- b   -> drop("beta")  -> free heap
      DEAD --- a   (already dead, drop skipped)
```

COMPILER OPTIMIZATION: NLL (Non-Lexical Lifetimes)

```
      Old borrow checker (pre-NLL):
      r_a live until:  }
      r_b live until:  }  end of enclosing scope
      a   live until:  }

      NLL borrow checker:
      r_a live until:  last use before println
      r_b live until:  last use before println
      a   live until:  last use = println!("{a}")
```

---

4. BORROW CHECKER: Access Control Decision Tree



SUMMARY TABLE:

```
+-----------------+--------------+--------------+--------------+
|                 |  Immutable   |  Mutable     |  Moved       |
|                 |  Borrow      |  Borrow      |              |
+-----------------+--------------+--------------+--------------+
| Can read?       |  YES         |  YES         |  NO          |
| Can write?      |  NO          |  YES         |  NO          |
| Can move?       |  NO          |  NO          |  NO          |
| Count allowed   |  Unlimited   |  1           |  N/A         |
| Concurrent with |  Other reads |  Nothing     |  N/A         |
| Lifespan        |  Until last  |  Until last  |  Permanent   |
|                 |  use         |  use         |  (transferred)|
+-----------------+--------------+--------------+--------------+
```

---

5. THREAD SAFETY: Cross-Domain Access Topology

```
DOMAIN A                    DOMAIN B                    DOMAIN C
(Thread 1)                  (Thread 2)                  (Thread 3)
+------------------+       +------------------+       +------------------+
|  Send + Sync     |       |  Send + Sync     |       |  Send + Sync     |
|  Data: i32       |       |  Data: i32       |       |  Data: i32       |
|  Clearance: HIGH |       |  Clearance: HIGH |       |  Clearance: HIGH |
+--------+---------+       +--------+---------+       +--------+---------+
         |                          |                          |
         |    +---------------------+---------------------+    |
         |    |                     |                     |    |
         |    |    SHARED RESOURCE DOMAIN                 |    |
         |    |    +-----------------------------+        |    |
         |    |    |  Arc<Mutex<Data>>           |        |    |
         |    |    |  Access Policy:             |        |    |
         |    |    |  - Lock() acquires permit   |        |    |
         |    |    |  - 1 writer XOR N readers   |        |    |
         |    |    |  - Automatic release on drop|        |    |
         |    |    +-----------------------------+        |    |
         |    +----------------+------------------------+    |
         |                     |                             |
         V                     V                             V
   +----------+          +----------+               +----------+
   | mpsc     |          | mpsc     |               | mpsc     |
   | channel  |--------->| channel  |-------------->| channel  |
   | tx       |          | rx/tx    |               | rx       |
   +----------+          +----------+               +----------+
```

SECURITY CLEARANCE HIERARCHY:

```
      +----------------------------------+
      |  ALL TYPES (default)             |
      |  +----------------------------+  |
      |  |  Send (cross-thread move)  |  |
      |  |  +----------------------+  |  |
      |  |  |  Sync (cross-thread  |  |  |
      |  |  |  shared access)      |  |  |
      |  |  |  +----------------+  |  |  |
      |  |  |  |  Arc<T>:       |  |  |  |
      |  |  |  |  T: Send+Sync  |  |  |  |
      |  |  |  |  = Thread safe |  |  |  |
      |  |  |  +----------------+  |  |  |
      |  |  +----------------------+  |  |
      |  +----------------------------+  |
      +----------------------------------+
```

TYPES THAT OPT OUT:

```
      +-------------+---------------+------------------+
      |  Type       |  Missing Trait|  Why             |
      +-------------+---------------+------------------+
      |  Rc<T>      |  Not Send     |  Non-atomic ref  |
      |             |               |  count would     |
      |             |               |  corrupt across  |
      |             |               |  threads         |
      +-------------+---------------+------------------+
```

```
     |  RefCell<T> |  Not Sync     |  Runtime borrow   |
     |             |               |  check without    |
     |             |               |  atomic guard     |
     +-------------+---------------+-------------------+
     |  *mut T     |  Not Send     |  Raw pointer has  |
     |             |  Not Sync     |  no safety        |
     |             |               |  guarantees       |
     +-------------+---------------+-------------------+
```

---

6. TYPE-STATE MACHINE: PhantomData & Zero-Cost Abstractions

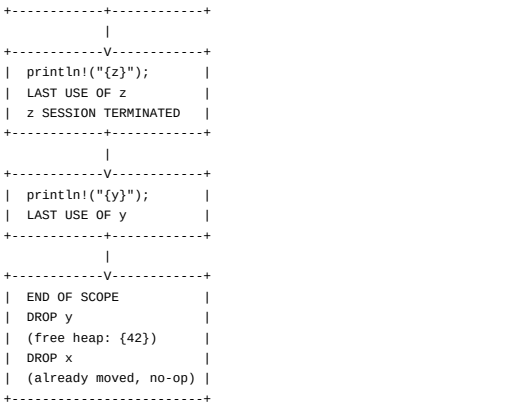
```
TYPE-STATE PATTERN (e.g., GPIO Pin in embedded Rust):

                    +--------------+
                    |  Pin<Disabled>|
                    +------+-------+
                           | .into_input()
                           V
                    +--------------+
                    |  Pin<Input>  |
                    +------+-------+
                           | .into_output()
                           V
                    +--------------+
                    | Pin<Output>  |
                    +------+-------+
                           | .set_high()
                           V
                    +--------------+
                    | Pin<Output>  |
                    | (high state) |
                    +--------------+
```

```
   struct Pin<MODE> {
       pin: u8,
       _mode: PhantomData<MODE>,
   }

   impl Pin<Output> {
       fn set_high(&self) { /* safe - compiler
                               guarantees Output mode */ }
   }

   // Compile-time state machine:
   // Call set_high() on Pin<Input> = COMPILE ERROR
   // Type system enforces state transitions
   // Zero runtime cost - PhantomData is zero-sized
```

---

7. OWNERSHIP LIFECYCLE: Flow Chart

```
                    +-------------------------+
                    |  let x = Box::new(42);  |
                    |  OWNER: x               |
                    |  Heap: {42}             |
                    +------------+------------+
                                 |
                    +------------V------------+
                    |  let y = x;             |
                    |  MOVE TRANSFER          |
                    |  OWNER: y               |
                    |  x: INVALIDATED         |
                    +------------+------------+
                                 |
                    +------------V------------+
                    |  let z = &y;            |
                    |  BORROW SESSION         |
                    |  OWNER: y               |
                    |  z: READ-ONLY           |
```

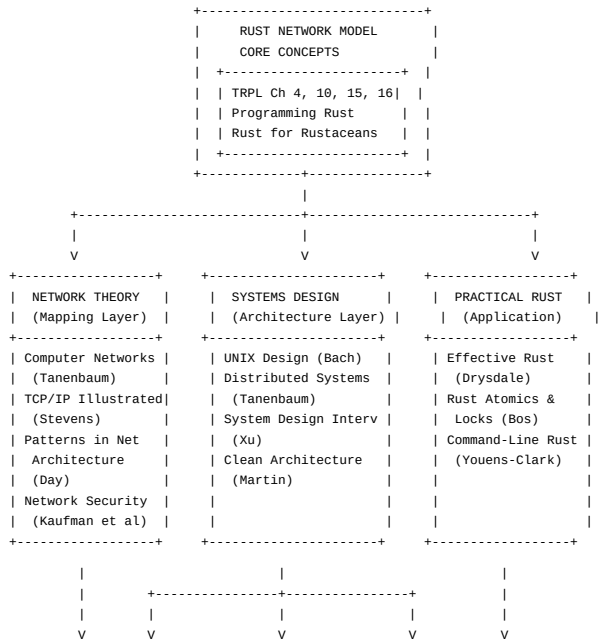


QUICK REFERENCE: OWNERSHIP STATES

Operation	Can Read	Can Write	Can Move
Owner (alive)	YES	YES	YES
Owner (dead/moved)	NO	NO	NO
Immut borrow	YES	NO	NO
Mut borrow	YES	YES	NO
After mut drop	YES (owner)	YES (owner)	YES (owner)

---

8. REFERENCE ARCHITECTURE: Books & Resources Network Diagram



```
   +-------------------------------------------------------------+
   |                    CROSS-CUTTING CONCEPTS                   |
   |  +------------+ +----------+ +----------+ +--------------+ |
   |  | Ownership  | |Lifetimes | |Borrowing | |  Concurrency | |
   |  | = Packet   | |= TTL     | |= Session | |  = Mesh Net | |
   |  +------------+ +----------+ +----------+ +--------------+ |
   |  +------------+ +----------+ +----------+ +--------------+ |
   |  | Send/Sync  | |Scopes    | | Traits   | |  Macros      | |
   |  |= Clearance | |= Subnets| |= Protocol| |  = Packet Gen| |
   |  +------------+ +----------+ +----------+ +--------------+ |
   +-------------------------------------------------------------+
```

---

9. QUICK REFERENCE CARD: Topology Map

```
+-------------------------------------------------------------------------------+
|                    RUST CONCEPT -> NETWORK EQUIVALENT                         |
+-------------------------------------------------------------------------------+
|                                                                               |
|  DATA UNIT        |  Rust                          |  Network                 |
|-------------------+--------------------------------+--------------------------|
|  Allocation       |  let x = T                     |  Packet enters zone      |
|  Move             |  let y = x                     |  Packet routed, orig     |
|                   |                                |  invalidated             |
|  Borrow (read)    |  let r = &x                    |  Read-only session       |
|  Borrow (write)   |  let r = &mut x                |  Exclusive write sess    |
|  Scope            |  { }                           |  Zone / subnet           |
|  Function call    |  f(x)                          |  Route traversal         |
|  Return           |  -> T                          |  Packet egress           |
|  Lifetime         |  'a                            |  TTL                     |
|  Static lifetime  |  'static                       |  Backbone permanence     |
|  Elision          |  (no lifetime written)         |  Default TTL negotiation |
|-------------------+--------------------------------+--------------------------|
|  CHECKING         |                                |                          |
|-------------------+--------------------------------+--------------------------|
|  Borrow checker   |  compiler pass                 |  Firewall / ACL engine   |
|  NLL              |  last-use tracking             |  Session timeout         |
|  Send trait       |  thread-safe move              |  Cross-domain clearance  |
|  Sync trait       |  thread-safe shared            |  Shared-domain clear.    |
|-------------------+--------------------------------+--------------------------|
|  CONCURRENCY      |                                |                          |
|-------------------+--------------------------------+--------------------------|
|  Thread           |  std::thread                   |  Network node            |
|  Channel          |  mpsc::channel                 |  Point-to-point link     |
|  Arc<T>           |  shared ownership              |  Replicated cache        |
|  Mutex<T>         |  mutual exclusion              |  Network lock / switch   |
|  RwLock<T>        |  read-write lock               |  Readers-writer policy   |
|-------------------+--------------------------------+--------------------------|
|  ABSTRACTION      |                                |                          |
|-------------------+--------------------------------+--------------------------|
|  Trait            |  interface specification       |  Protocol contract       |
|  dyn Trait        |  runtime dispatch              |  Dynamic routing         |
|  Generics         |  <T: Bound>                    |  Policy-compliant pkt    |
|  impl Trait       |  opaque return type            |  Encapsulated PDU        |
|  PhantomData      |  type-level state              |  Protocol tag            |
|-------------------+--------------------------------+--------------------------|
|  MEMORY           |                                |                          |
|-------------------+--------------------------------+--------------------------|
|  Box<T>           |  heap allocation               |  Remote storage          |
|  Rc<T>            |  ref count (single thread)     |  Local multicast         |
|  RefCell<T>       |  runtime borrow (single thread)|  Local lock              |
|  Unsafe           |  unchecked operations          |  Raw socket access       |
|-------------------+--------------------------------+--------------------------|
|  ERROR HANDLING   |                                |                          |
|-------------------+--------------------------------+--------------------------|
|  Result<T,E>      |  fallible operation            |  ACK/NACK protocol       |
|  ? operator       |  propagate error               |  NACK propagation        |
|  unwrap()         |  assume success                |  Assume ACK (risky)      |
|  panic!           |  unrecoverable error           |  Node failure            |
+-------------------------------------------------------------------------------+
```

---

10. TERMINOLOGY ARCHITECT'S GLOSSARY

NEW TERM	INSTEAD OF	ARCHITECTURAL MEANING
Data zone	variable scope	Bounded territory
Packet custody	ownership	Exclusive control
Route / forward	move	Transfer action
Session	borrow	Temporary access
Exclusive write sess	mutable borrow	Write access mode
Read-only session	immutable borrow	Read access mode
TTL / session dur.	lifetime	Time-bounded validity
Default TTL neg.	lifetime elision	Auto protocol
Access firewall	borrow checker	Policy enforcement
Compile-time ACL	borrow checker	Pre-deployment check
Node / exec. domain	thread	Independent agent
Link / circuit	channel	Network connection
Cross-domain clearance	Send trait	Security policy
Shared-domain clear.	Sync trait	Concurrent access
Replicated cache	Arc<T>	Distributed access
Network lock	Mutex<T>	Exclusive access
Route traversal	function call	Boundary crossing
Egress packet	return value	Directional flow
NACK propagation	? operator	Failure signaling
Node failure	panic!	Catastrophic event
Packet schema	type system	Data format
Protocol compliance	trait bound	Interface contract
Pre-deployment verify	compile time	Safety guarantee

Appendix: RFC 1925 - The Twelve Networking Truths (Applied to Rust)

RFC 1925 "The Twelve Networking Truths" - Ross Callon, 1996

+---- Truth 1 ----------
| "It Has To Work."
| -> Rust: The borrow checker guarantees safety. If it compiles,
| there are no data races, use-after-frees, or dangling pointers.
+---

+---- Truth 2 ----------
| "No matter how hard you push the envelope, it remains stationery."
| -> Rust: Zero-cost abstractions - no matter how high-level your
| code, the compiled result is as efficient as hand-written C.
+---

+---- Truth 6 ----------
| "It is always possible to add another level of indirection."
| -> Rust: Box<dyn Trait>, Rc<RefCell<T>>, Arc<Mutex<T>> - each
| layer adds network-grade indirection with a purpose.
+---

+---- Truth 7a ----------
| (corollary) "... but indirection can hide problems."
| -> Rust: Unsafe code hides safety issues. Arc<Mutex<T>> hides
| lock contention. Measure before optimizing.
+---

+---- Truth 8 ----------
| "It is more important to have the right interface than to have the
| right implementation."
| -> Rust: Traits define protocol contracts. A good trait design
| outlives any single implementation.
+---

+---- Truth 10 ----------
| "One size rarely fits all."
| -> Rust: Choose ownership (unicast), Arc (multicast), channels
| (point-to-point), or Mutex (shared link) - each data flow
| needs the right topology.
+---

+---- Truth 12 ----------

```
  |  "In protocol design, perfection has been reached not when there    |
  |   is nothing left to add, but when there is nothing left to take    |
  |   away."                                                            |
  |  -> Rust: Minimal, orthogonal rules (ownership, borrowing, lifetimes)|
  |    compose to produce maximum safety. Every feature pulls its       |
  |    weight.                                                          |
  +---------------------------------------------------------------------+
```

"Rust is what happens when a network architect designs a programming language - every data flow is a routed packet, every borrow is a session, every scope is a subnet, and the compiler is the firewall that keeps it all safe."